Chapter 1
Decision Making and Looping

INTRODUCTION

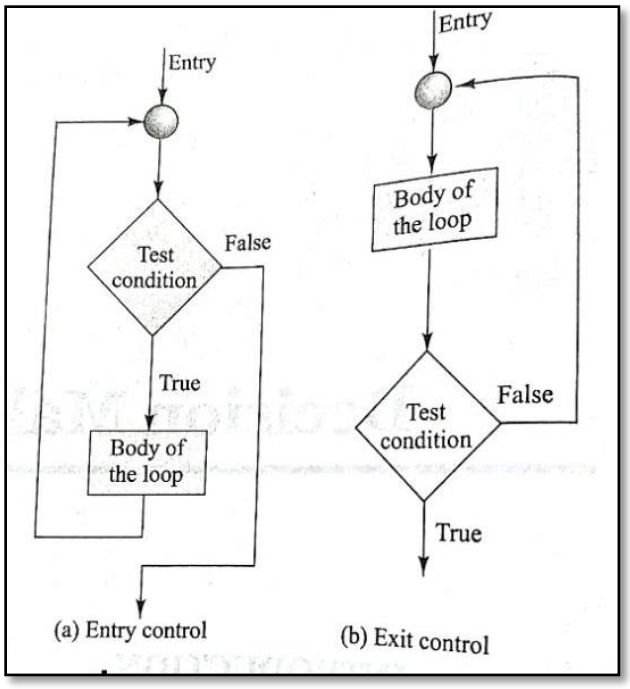
A computer is well suited to perform repetitive operations. Every computer language must have features that instruct a computer to perform such repetitive tasks. The process of repeatedly executing a block of statements is known as looping. The statements in the block may be executed any number of times, from zero to infinite number. If a loop continues forever, it is called an **infinite loop**.
In looping, sequences of statements areexecuted until some conditions for the termination of the loop are satisfied. A **program loop** therefore consists of two segments, one known as the **body of the loop** and the other known as the **control statement**. The control statement tests certain conditions and then directs the repeated execution of the statements contained in the body of the loop
Depending on the position of the control statement in the loop, a control structure may be classified either as the **entry controlled** loop or as **exit controlled** loop. The below flowcharts illustrate these structures. In the entry controlled loop, the control conditions are tested before the start of the loop execution. If the conditions are not satisfied, then the body of the loop will not be executed. In the case of an exit controlled loop, the test is performed at the end of the body of the loop and therefore the body is executed unconditionally for the first time.
The looping process includes the following 4 steps:

1. Setting and initialization of a counter
2. Execution of the statements in the loop
3. Test for a specified condition for execution of the loop
4. Incrementing the counter

The test may be either to determine whether the loop has been repeated the specified number of times or to determine whether a particular condition has been met with.

The java language provides for three constructs for performing loop operations. They are:

1. **while** construct
2. **do** construct
3. **for** construct



THE while STATEMENT

The simplest of all the looping structures in Java is the while statement. The basic format of the while statement is:

```
Initialization;
while(test condition)
{
     Body of the loop
}
```

The **while** is an entry controlled loop statement. The test condition is evaluated and if the condition is true, then the body of the loop is executed. After execution of the body, the test condition is once again evaluated and if it is true, the body is executed once again. This process of repeated execution of the body continues until the test condition finally becomes false and the control is transferred out of the loop. On exit, the program continues with the statement immediately after the body of the loop.

The body of the loop may have one or more statements. The braces are needed only if the body contains two or more statements. However, it is a good practice to use braces even if the body has only one statement.

Consider the flowing code segment:

.....

.....

```
sum=0;
```

```
n=1;
while(n<=10)
{
     sum=sum+n*n;
     n=n+1;
}
System.out.println("sum="+sum);
```

……

The body of the loop is executed 10 times for n=1, 2,….,10 each time adding the square of the value of n, which is incremented inside the loop. The test condition may also be written as n<11; the result would be the same.

The example program is given below:

```
// Demonstrate the while loop.
class While {
  public static void main(String args[]) {
    int n = 10;

    while(n > 0) {
      System.out.println("tick " + n);
      n--;
    }
  }
}
```

The output of the program is

```
tick 10
tick 9
tick 8
tick 7
tick 6
tick 5
tick 4
tick 3
tick 2
tick 1
```

THEdo STATEMENT

The **while** loop construct that we discussed in the previous section makes a test condition before the loop is executed. Therefore, the body of the loop may not be executed at all if the condition is not satisfied at the very first attempt. On some occasions it might be necessary to execute the body of the loop before the test is

performed. Such situations can be handled with the help of the **do** statement. This takes the form:

```
Initialization;
do
{
     Body of the loop
}
while(test condition);
```

On reaching the **do** statement, the program proceeds to evaluate the body of the loop first. At the end of the loop, the test condition in the **while** statement is evaluated. If the condition is true, the program continues to evaluate the body of the loop once again. This process continues as long as the condition is true. When the condition becomes false, the loop will be terminated and the control goes to the statement that appears immediately after the **while** statement

Since the test condition is evaluated at the bottom of the loop, the do…while construct provides an exit controlled loop and therefore the body of the loop is always executed at least once.

Consider the example:

```
.....
.....
i=1;
sum=0;
do
{
     sum=sum+1;
     i=i+2;
}
while(sum<40 || i<10);
.....
```

The loop will be executed as long as one of the two relations is true.

The example program:

```
// Demonstrate the do-while loop.
class DoWhile {
  public static void main(String args[]) {
    int n = 10;
```

```
    do {
      System.out.println("tick " + n);
      n--;
    } while(n > 0);
  }
}
```

The output:

```
tick 10
tick 9
tick 8
tick 7
tick 6
tick 5
tick 4
tick 3
tick 2
tick 1
```

The below table lists the difference between while and do-while loops

While	do-while
It is a looping construct that will execute only f the test condition is true	It is a looping construct that will execute at least once even if the test condition is false
It is an entry-controlled loop	It is an exit-controlled loop
It is generally used for implementing common looping situations	It is typically used for implementing menu based programs where the menu is required to be printed at least once

THE for STATEMENT

The for loop is another entry controlled loop that provides a more concise loop control structure. The general form of the **for** loop is

```
for(initialization ; test condition ; increment)
{
     Body of the loop
}
```

The execution of the **for** statement is as follows:

1. Initialization of the control variables is done first, using assignment statements such as i=1 and count=0. The variables**i** and **count** are known as loop-control variables

2. The value of the control variable is tested using 10test condition. The test condition is a relational expression, such as i<10 that determines when the loop will exit. If the condition is true, the body of the loop is executed; otherwise the loop is terminated and the execution continues with the statement that immediately follows the loop.
3. When the body of the loop is executed, the control is transferred back to the **for** statement after evaluating the last statement in the loop. Now, the control variable is incremented using an assignment statement such as i=i+1 and the new value of the control variable is again tested to see whether it satisfies the loop condition. If the condition is satisfied, the body of the loop is again executed. This process continues till the value of the control variable fails to satisfy the test condition.

Consider the following segment of a program

```
for(x=0;x<=9;x=x+1)
{
     System.out.println(x);
}
```

This **for** is executed 10 times and prints the digits 0 to 9 in one line. The three sections enclosed within parenthesis must be separated by semicolons. There is no semicolon at the end of the increment section, **x=x+1**

The **for**statement allows for negative increments. For example, the loop discussed above can be written as follows:

```
for(x=9;x>=0;x=x-1)
{
     System.out.println(x);
}
```

This loop is also executed 10 times, but the output would be from 9 to 0 instead of 0 to 9.

Since the conditional test is always performed at the beginning of the loop, the body of the loop may not be executed at all, if the condition fails at the start. For example,

```
for(x=9;x<9;x=x-1)
{
     .....
     .....
}
```

Will never be executed because the test condition fails at the very beginning itslef. The example program:

```
// Demonstrate the for loop.
class ForTick {
  public static void main(String args[]) {
    int n;
    for(n=10; n>0; n--)
      System.out.println("tick " + n);
  }
}
```

Additional Features of for Loop

The **for** loop has several capabilities that are not found in other loop constructs. For example, more than one variable can be initialized at a time in the for statement. The statements

```
p=1;
for(n=0;n<17;++n)
```

can be rewritten as

```
for(p=1,n=0;n<17;++n)
```

Notice that the initialization section has 2 parts p=1 and n=0 separated by a comma.

Like the initialization section, the increment section may also have more than one part. For example, the loop

```
for(n=1,m=50;n<=m;n=n+1,m=m-1)
{
      .....
      .....
}
```

Is perfectly valid. The multiple arguments in the increment section are separated by commas.

The third feature is that the test condition may have any compound relation and the testing need not be limited only to the loop control variable. Consider the example that follows:

```
sum=0;
for(i=1;i<20 && sum<100;++i)
{
      .....
      .....
}
```

The loop uses a compound test condition with the control variable **i**and external variable **sum**. The loop is executed as long as both the conditions i<20 and sum<100 are true. The sum is evaluated inside the loop.

Another unique aspect of **for** loop is that one or more sections can be omitted, if necessary. Consider the following statements:

```
……
……
m=5;
for(;m!=100;)
{
     System.out.println(m);
     m=m+5;
}
……
……
```

Both the initialization and increment sections are omitted in the **for**statement. The initialization is done before the **for**statement and the control variable is incremented inside the loop. In such cases, the sections are left blank. However, the semicolons separating the sections must remain. If the test condition is not present, the **for** statement sets up an infinite loop

We can set up time delay loops using the null statement as follows:

```
for(j=1000;j>0;j=j-1)
     ;
```

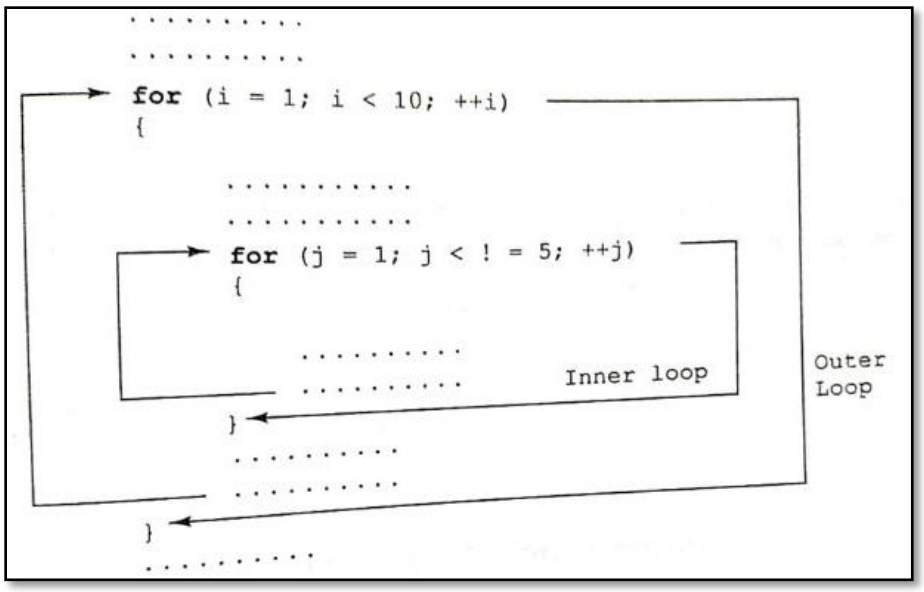
This loop is executed 1000 times without producing any output; it simply causes a time delay. Notice that the body of the loop contains only a semicolon, known as an empty statement. This can also be written as

```
for(j=1000;j>0;j=j-1);
```

This implies that the compiler will not give an error message if we place a semicolon by mistake at the end of a **for**statement. The semicolon will be considered as an empty statement.

Nesting of for Loops

Nesting of loops, that is, one for statement within another **for**statement is allowed in Java.



The loops should be properly indented so as to enable the reader to easily determine which statements are contained within each for statement

```
// Loops may be nested.
class Nested {
  public static void main(String args[]) {
    int i, j;

    for(i=0; i<10; i++) {
      for(j=i; j<10; j++)
        System.out.print(".");
      System.out.println();
    }
  }
}
```

The output of the program:

```
- - - - - - - - - -
- - - - - - - - -
- - - - - - - -
- - - - - - -
- - - - -
- - - -
- - -
- -
-
```

The Enhanced for Loop

The enhanced for loop is also called as for each loop, is an extended language feature introduced with the J2SE 5.0 release. This feature helps us to retrieve the array of elements efficiently rather than using array indexes. We can also use this feature to eliminate the iterators in a **for** loop and to retrieve the elements from a collection. The enhanced for loop takes the following form:

```
for(Type Identifier:Expression)
{
//statements;
}
```

Where, **Type** represents the data type or object used; **Identifier** refers to the name of a variable; and **Expression** is an instance of the java.lang.Iterable interface or an array.

```
// Use a for-each style for loop.
class ForEach {
  public static void main(String args[]) {
    int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    int sum = 0;

    // use for-each style for to display and sum the values
    for(int x : nums) {
      System.out.println("Value is: " + x);
      sum += x;
    }

    System.out.println("Summation: " + sum);
  }
}
```

Output:

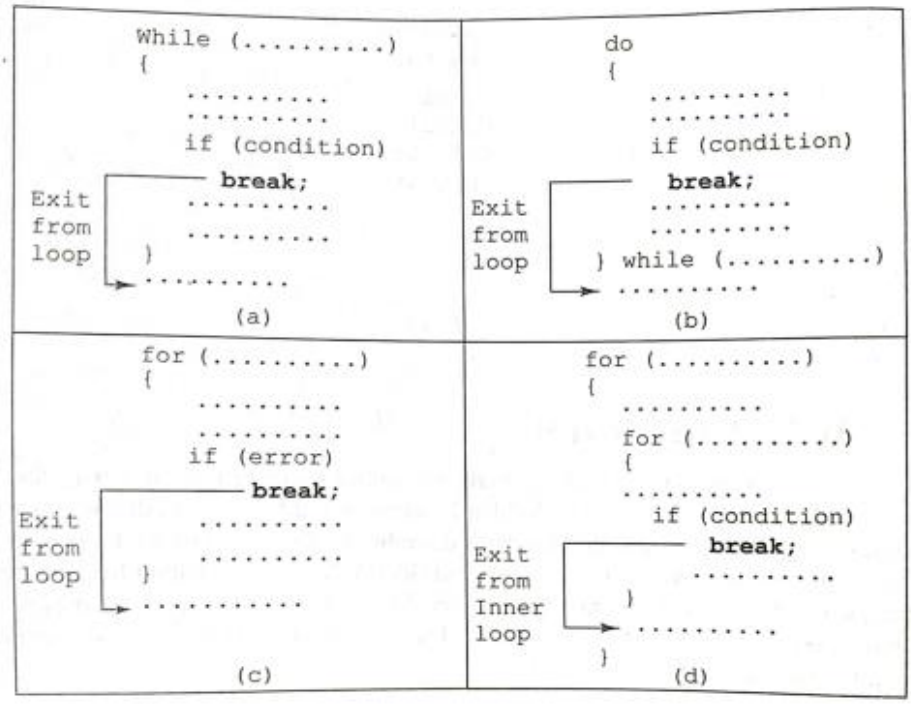
```
Value is: 1
Value is: 2
Value is: 3
Value is: 4
Value is: 5
Value is: 6
Value is: 7
Value is: 8
Value is: 9
Value is: 10
Summation: 55
```

JUMPS IN LOOPS

Loops perform a set of operations repeatedly until the control variable fails to satisfy the test condition. The number of times a loop is repeated is decided in advance and the test condition is written to achieve this. Sometimes, when executing a loop it becomes desirable to skip a part of the loop or to leave the loop as soon as a certain condition occurs. For example, consider the case of searching for a particular name in a list containing, say, 100 names. A program loop written for reading and testing the names a 100 times must be terminated as soon as the desired name is found. Java permits a jump from one statement to the end or beginning of a loop as well as a jump out of a loop.

Jumping Out of a Loop

An early exit from a loop can be accomplished by using the **break** statement. The **break** is used with **switch** statement. This statement can also be used within **while**, **do** or **for** loops



When the **break** statement is encountered inside a loop, the loop is immediately exited and the program continues with the statement immediately following the loop. When the loops are nested, the break would only exit from the loop containing it. That is, the break will exit only a single loop.

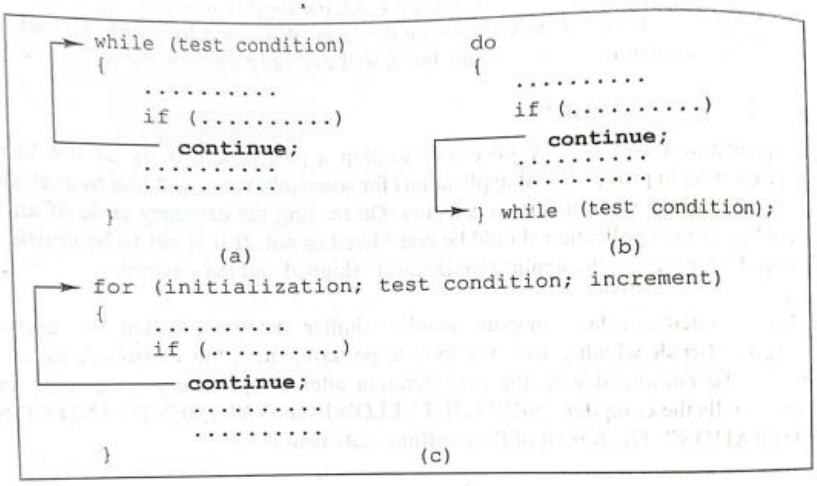
Skipping a Part of a Loop

During the loop operations, it may be necessary to skip a part of the body of the loop under certain conditions. Like the break statement, Java supports another similar statement called the **continue** statement. However, unlike the **break** which causes the loop to be terminated, the **continue**, as the implies causes the loop to be

continued with the next iteration after skipping any statements in between. The continue statement tells the compiler "SKIP THE FOLLOWPING STATEMENT AND CONTINUE WITH THE NEXT ITERATION". The format of the continue statement is simply

```
continue;
```

The use of the **continue** statement in loops is illustrated in below figure



In **while** and **do** loops, continue causes the control to go directly to the test condition and then continue the iteration process. In the case of **for** loop, the increment section of the loop is executed before the test condition is evaluated.

LABELLED LOOPS

In Java, we can give a label to a block of statements. A label is any valid Java variable name. To give a label to a loop, place it before the loop with a colon at the end. Example:

```
loop1:   for (.....)
         {
             ................
             ................
         }
         ................
```

A block of statements can be labeled as shown below:

```
block1:  {
              ..........
              ..........
block2:       {
                   ..........
                   ..........
              }
              ..........
              ..........
         }
```

We have seen that a simple **break** statement causes the control to jump outside the nearest loop and a simple **continue** statement restarts the current loop. If we want to jump outside a nested loops or to continue a loop that is outside the current one, then we may have to use the labeled **break** and labeled **continue** statements.
Example:

```
outer:  for (int m = 1; m<11; m++)
        {
           for (int n = 1; n<11; n++)
           {
              System. out. print (" " + m*n);
              if (n == m)
                 continue outer;
           }
        }
```

Here, the **continue** statement terminated the inner loop when n=m and continues with the next iteration of the outer loop (counting m).
Another example,

```
loop1:   for (int  i = 0; i < 10; i++)
         {
loop2:      while (x < 100)
            {
               Y = i * x;
               if (Y > 500)
                  break loop1;
               ..........
               ..........
            }
            ..........
            ..........
         }
         ..........
```

LOOP1 LOOP2

Jumping out of both loops

Here, the label **loop1** labels the outer loop and therefore the statement

```
break loop1;
```
Causes the execution to break out of both the loops.

The example program for break:

```
classGFG {
    publicstaticvoidmain(String[] args)
    {
        // Initially loop is set to run from 0-9
        for(inti = 0; i < 10; i++) {
            // Terminate the loop when i is 5
            if(i == 5)
                break;
            System.out.println("i: "+ i);
        }
        System.out.println("Out of Loop");
    }
}
```

The Output is:

```
i: 0
i: 1
i: 2
i: 3
i: 4
Out of Loop
```

Example program for labeled break:

```
classGFG {
    publicstaticvoidmain(String args[])
    {
    // First label
    first:
        for(inti = 0; i < 3; i++) {
        // Second label
        second:
            for(intj = 0; j < 3; j++) {
                if(i == 1&& j == 1) {

                    // Using break statement with label
                    breakfirst;
                }
```

```
                System.out.println(i + " "+ j);
            }
        }
    }
}
```

Output

```
0 0
0 1
0 2
1 0
```

Example program for continue:

```
classGFG {
    publicstaticvoidmain(String args[])
    {
        for(inti = 0; i < 10; i++) {
            // If the number is 2
            // skip and continue
            if(i == 2)
                continue;

            System.out.print(i + " ");
        }
    }
}
```

Output

```
0 1 3 4 5 6 7 8 9
```

Example program for labeled continue:

```
classGFG {
    publicstaticvoidmain(String args[])
    {
    // First label
    first:
        for(inti = 0; i < 3; i++) {
        // Second label
        second:
            for(intj = 0; j < 3; j++) {
                if(i == 1&& j == 1) {
```

```
                    // Using continue statement with label
                    continuefirst;
                }
                System.out.println(i + " "+ j);
            }
        }
    }
}
```

Output

```
0 0
0 1
0 2
1 0
2 0
2 1
2 2
```

Chapter 2
Classes and Objects

CLASS FUNDAMENTALS

A class is that it defines anew data type. Once defined, this new type can be used to create objects of that type. Thus, a class is a *template* for an object, and an object is an *instance* of a class. Because an object is an instance of a class, you will often see the two words *object*and *instance* used interchangeably.

The General Form of a Class

When you define a class, you declare its exact form and nature. You do this by specifying the data that it contains and the code that operates on that data. While very simple classes may contain only code or only data, most real-world classes contain both.

A class is declared by use of the **class** keyword. A simplified general form of a **class**definition is shown here:

```
class classname {
   type instance-variable1;
   type instance-variable2;
   // ...
   type instance-variableN;

   type methodname1(parameter-list) {
     // body of method
   }
   type methodname2(parameter-list) {
     // body of method
   }
   // ...
   type methodnameN(parameter-list) {
      // body of method
   }
}
```

The data, or variables, defined within a **class** are called *instance variables*. The code is contained within *methods*. Collectively, the methods and variables defined within a class are called *members* of the class. In most classes, the instance variables are acted upon and accessed by the methods defined for that class. Variables defined within a class are called instance variables because each instance of the class (that is, each object of the class) contains its own copy of these variables.

All methods have the same general form as **main( )**. However, most methods will not be specified as **static** or **public**. Notice that the general form of a class does not specify a **main( )** method. Java classes donot need to have a **main()**method. Some kinds of Java applications don't require a**main( )** method at all.

A Simple Class

Here is a class called **Box** that defines three instance variables: **width, height**, and **depth**. Currently, **Box** doesnot contain any methods

```
class Box {
  double width;
  double height;
  double depth;
}
```

As stated, a class defines a new type of data. In this case, the new data type is called **Box**. You will use this name to declare objects of type **Box**. It is important to remember that a class declaration only creates a template; it does not create an

actual object. Thus, the preceding code does not cause any objects of type **Box** to come into existence.
To actually create a **Box** object, you will use a statement like the following:

```
Box mybox = new Box(); // create a Box object called mybox
```

Each time you create an instance of a class, you are creatingan object that contains its own copy of each instance variable defined by the class.
Thus, every **Box** object will contain its own copies of the instance variables **width, height**, and **depth**. To access these variables, you will use the *dot* (.) operator. Thedot operator links the name of the object with the name of an instance variable. Forexample, to assign the **width** variable of **mybox**the value 100, you would use thefollowing statement:

```
mybox.width = 100;
```

This statement tells the compiler to assign the copy of **width** that is contained within the **mybox**object the value of 100. In general, you use the dot operator to access both the instance variables and the methods within an object.
Here is a complete program that uses the **Box** class:

```
/* A program that uses the Box class.

   Call this file BoxDemo.java
*/
class Box {
  double width;
  double height;
  double depth;
}

// This class declares an object of type Box.
class BoxDemo {
  public static void main(String args[]) {
    Box mybox = new Box();
    double vol;

    // assign values to mybox's instance variables
    mybox.width = 10;
    mybox.height = 20;
    mybox.depth = 15;

    // compute volume of box
    vol = mybox.width * mybox.height * mybox.depth;

    System.out.println("Volume is " + vol);
  }
}
```

You should call the file that contains this program **BoxDemo.java**, because the **main( )** method is in the class called **BoxDemo**, not the class called **Box**. When you compile this program, you will find that two **.class** files have been created, one for **Box** and one for **BoxDemo**. The Java compiler automatically puts each class into its own **.class** file. It is not necessary for both the **Box** and the **BoxDemo**class to actually be in the same source file. You could put each class in its own file, called **Box.java** and **BoxDemo.java**, respectively.
To run this program, you must execute **BoxDemo.class**. When you do, you will see the following output:

```
Volume is 3000.0
```

As stated earlier, each object has its own copies of the instance variables. This means that if you have two **Box** objects, each has its own copy of **depth, width**, and **height**. It is important to understand that changes to the instance variables of one object have no effect on the instance variables of another. For example, the following program declares two **Box** objects:

```
// This program declares two Box objects.

class Box {
  double width;
  double height;
  double depth;
}

// This class declares an object of type Box.
class BoxDemo {
  public static void main(String args[]) {
    Box mybox = new Box();
    double vol;

    // assign values to mybox's instance variables
    mybox.width = 10;
    mybox.height = 20;
    mybox.depth = 15;

    // compute volume of box
    vol = mybox.width * mybox.height * mybox.depth;

    System.out.println("Volume is " + vol);
  }
}
```

The output produced by this program is shown here:

```
Volume is 3000.0
Volume is 162.0
```

As you can see, **mybox1**'s data is completely separate from the data contained in **mybox2**.

DECLARING OBJECTS

As just explained, when you create a class, you are creating a new data type. You can use this type to declare objects of that type. However, obtaining objects of a class is a two-step process. First, you must declare a variable of the class type. This variable does not define an object. Instead, it is simply a variable that can *refer* to an object. Second, you must acquire an actual, physical copy of the object and assign it to that variable. You can do this using the **new** operator. The **new** operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, essentially, the address in memory of the object allocated by **new**. This reference is then stored in the variable. Thus, in Java, all class objects must be dynamically allocated.

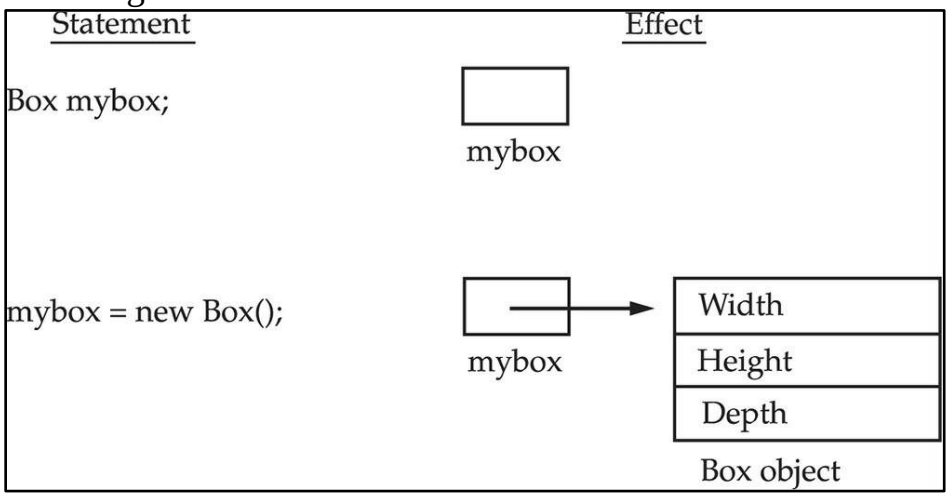
In the preceding sample programs, a line similar to the following is used to declare an object of type **Box**:

```
Box mybox = new Box();
```

This statement combines the two steps just described. It can be rewritten like this to show each step more clearly:

```
Box mybox; // declare reference to object
mybox = new Box(); // allocate a Box object
```

The first line declares **mybox**as a reference to an object of type **Box**. At this point,**mybox**does not yet refer to an actual object. The next line allocates an object and assigns a reference to it to **mybox**. After the second line executes, you can use **mybox**as if it were a **Box** object. The effect of these two lines of code isdepicted in below figure



A Closer Look at new

As just explained, the **new** operator dynamically allocates memory for an object. In the context of an assignment, it has this general form:

***class-var*= new *classname*( );**

Here, *class-var*is a variable of the class type being created. The *classmane*is the name of the class that is being instantiated. The class name followed by parentheses specifies the *constructor* for the class. A constructor defines what occurs when an object of a class is created. Constructors are an important part of all classes and have many significant attributes. Most real-world classes explicitly define their own constructors within their class definition. However, if no explicit constructor is specified, then Java will automatically supply a default constructor. This is the case with **Box**. For now, we will use the default constructor.

It is important to understand that **new** allocates memory for an object during run time. The advantage of this approach is that your program can create as many or as few objects as it needs during the execution of your program. However, since memory is finite, it is possible that **new** will not be able to allocate memory for an object because insufficient memory exists. If this happens, a run-time exception will occur.

Let's once again review the distinction between a class and an object. A class creates a new data type that can be used to create objects. That is, a class creates a logical framework that defines the relationship between its members. When you declare an object of a class, you are creating an instance of that class. Thus, a class is a logical construct. An object has physical reality. (That is, an object occupies space in memory.)

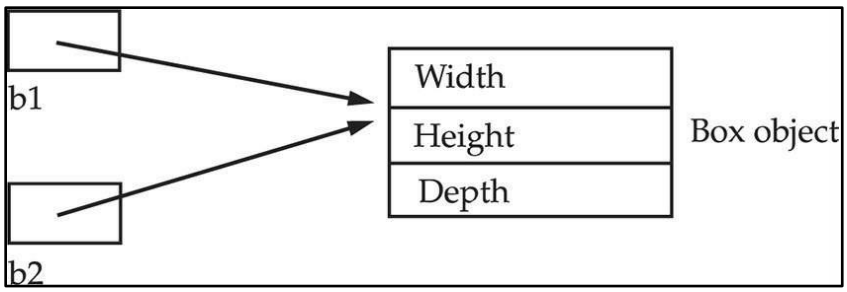
ASSIGNING OBJECT REFERENCE VARIABLES

Object reference variables act differently than you might expect when an assignment takes place. For example, what do you think the following fragment does?

```
Box b1 = new Box();
Box b2 = b1;
```

You might think that **b2** is being assigned a reference to a copy of the object referred to by **b1**. That is, you might think that **b1** and **b2** refer to separate and distinct objects. However, this would be wrong. Instead, after this fragment executes, **b1** and **b2** will both refer to the *same* object. The assignment of **b1** to **b2** did not allocate any memory or copy any part of the original object. It simply makes **b2** refer to the same object as does**b1**. Thus, any changes made to the object through **b2** will affect the object to which **b1** is referring, since they are the same object.

This situation is depicted here:



INTRODUCING METHODS

Classes usually consist of two things: instance variables and methods. This is the general form of a method:

```
type name(parameter-list) {
    // body of method
}
```

Here, *type* specifies the type of data returned by the method. This can be any valid type, including class types that you create. If the method does not return a value, its return type must be **void**. The name of the method is specified by *name*. This can be any legal identifier other than those already used by other items within the current scope. The *parameter-list* is a sequence of type and identifier pairs separated by commas. Parameters are essentially variables that receive the value of the arguments passed to the method when it is called. If the method has no parameters, then the parameter list will be empty.

Methods that have a return type other than **void** return a value to the calling routine using the following form of the **return** statement:

```
return value;
```

Here, *value* is the value returned.

Adding a Method to the Box Class

Although it is perfectly fine to create a class that contains only data, it rarely happens. Most of the time, you will use methods to access the instance variables defined by the class. In fact, methods define the interface to most classes.

Let's begin by adding a method to the **Box** class.

```
// This program includes a method inside the box class.

class Box {
  double width;
  double height;
  double depth;

  // display volume of a box
  void volume() {
    System.out.print("Volume is ");
    System.out.println(width * height * depth);
  }
}

class BoxDemo3 {
  public static void main(String args[]) {
    Box mybox1 = new Box();
    Box mybox2 = new Box();

    // assign values to mybox1's instance variables
    mybox1.width = 10;
    mybox1.height = 20;
    mybox1.depth = 15;
    /* assign different values to mybox2's
       instance variables */
    mybox2.width = 3;
    mybox2.height = 6;
    mybox2.depth = 9;

    // display volume of first box
    mybox1.volume();

    // display volume of second box
    mybox2.volume();
  }
}
```

This program generates the following output, which is the same as the previous version.

```
Volume is 3000.0
Volume is 162.0
```

Look closely at the following two lines of code:

```
mybox1.volume();
mybox2.volume();
```

The first line here invokes the **volume( )** method on **mybox1**. That is, it calls **volume( )** relative to the **mybox1** object, using the object's name followed by the dot operator. Thus, the call to **mybox1.volume( )** displays the volume of the box defined by **mybox1**, and the call to **mybox2.volume( )** displays the volume of the box defined by **mybox2**. Each time **volume( )** is invoked, it displays the volume for the specified box.

When **mybox1.volume( )** is executed, the Java run-time system transfers control to the code defined inside **volume( )**. After the statements inside **volume( )** have executed, control is returned to the calling routine, and execution resumes with the line of code following the call.

There is something very important to notice inside the **volume( )** method: the instance variables **width, height**, and **depth** are referred to directly, without preceding them with an object name or the dot operator. When a method uses an instance variable that is defined by its class, it does so directly, without explicit reference to an object and without use of the dot operator.

When an instance variable is accessed by code that is not part of the class in which that instance variable is defined, it must be done through an object, by use of the dot operator. However, when an instance variable is accessed by code that is part of the same class as the instance variable, that variable can be referred to directly. The same thing applies to methods.

Returning a Value

What if another part of your program wanted to know the volume of a box, but not display its value? A better way to implement **volume( )** is to have it compute the volume of the box and return the result to the caller. The following example, an improved version of the preceding program, does just that:

```
// Now, volume() returns the volume of a box.

class Box {
  double width;
  double height;
  double depth;

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
class BoxDemo4 {
  public static void main(String args[]) {
    Box mybox1 = new Box();
    Box mybox2 = new Box();
    double vol;

    // assign values to mybox1's instance variables
    mybox1.width = 10;
    mybox1.height = 20;
    mybox1.depth = 15;

    /* assign different values to mybox2's
       instance variables */
    mybox2.width = 3;
    mybox2.height = 6;
    mybox2.depth = 9;

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

As you can see, when **volume( )** is called, it is put on the right side of an assignment statement. On the left is a variable, in this case **vol**, that will receive the value returned by **volume( )**.

```
vol = mybox1.volume();
```

There are two important things to understand about returning values:

- The type of data returned by a method must be compatible with the return type specified by the method. For example, if the return type of some method is **boolean**, you could not return an integer.
- The variable receiving the value returned by a method (such as **vol**, in this case) must also be compatible with the return type specified for the method.

Adding a Method That Takes Parameters

While some methods don't need parameters, most do. Parameters allow a method to be generalized. That is, a parameterized method can operate on a variety of data.

To illustrate this point, let's use a very simple example. Here is a method that returns the square of thenumber 10:

```
int square()
{
  return 10 * 10;
}
```

While this method does, indeed, return the value of 10 squared, its use is very limited. However, if you modify the method so that it takes a parameter, as shown next, then you can make **square( )** much more useful.

```
int square(int i)
{
  return i * i;
}
```

Now, **square( )** will return the square of whatever value it is called with. That is, **square( )** is now a general-purpose method that can compute the square of any integer value, rather than just 10.

Here is an example:

```
int x, y;
x = square(5); // x equals 25
x = square(9); // x equals 81
y = 2;
x = square(y); // x equals 4
```

In the first call to **square( )**, the value 5 will be passed into parameter **i**. In the second call, **i** will receive the value 9. The third invocation passes the value of **y**, which is 2 in this example. As these examples show, **square( )** is able to return the square of whatever data it is passed.

It is important to keep the two terms *parameter* and *argument* straight. A*parameter*is a variable defined by a method that receives a value when the method is called. For example, in **square( ), i** is a parameter. An *argument* is a value that is passed to a method when it is invoked. For example, **square(100)** passes 100 as an argument. Inside **square( )**, the parameter **i** receives that value.

You can use a parameterized method to improve the **Box** class.

This concept is implemented by the following program:

```
// This program uses a parameterized method.

class Box {
  double width;
  double height;
  double depth;

  // compute and return volume
  double volume() {
    return width * height * depth;
  }

  // sets dimensions of box
  void setDim(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }
}

class BoxDemo5 {
  public static void main(String args[]) {
    Box mybox1 = new Box();
    Box mybox2 = new Box();
    double vol;

    // initialize each box
    mybox1.setDim(10, 20, 15);
    mybox2.setDim(3, 6, 9);

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

As you can see, the **setDim( )** method is used to set the dimensions of each box. For example, when

```
mybox1.setDim(10, 20, 15);
```

is executed, 10 is copied into parameter **w**, 20 is copied into **h**, and 15 is copied into **d**. Inside **setDim( )** the values of **w, h**, and **d** are then assigned to **width, height**, and **depth**, respectively.

CONSTRUCTORS

Java allows objects to initialize themselves when they are created. This automatic initialization is performed through the use of a constructor.

A *constructor* initializes an object immediately upon creation. It has the same name as the class in which it resides and is syntactically similar to a method. Once defined, the constructor is automatically called when the object is created, before the **new** operator completes. Constructors look a little strange because they have no return type, not even **void**. This is because the implicit return type of a class' constructor is the class type itself.

You can rework the **Box** example so that the dimensions of a box areautomatically initialized when an object is constructed. To do so, replace **setDim( )**with a constructor. Let's begin by defining a simple constructor that sets the dimensions of each box to the same values. This version is shown here:

```
/* Here, Box uses a constructor to initialize the
   dimensions of a box.
*/
class Box {
  double width;
  double height;
  double depth;

  // This is the constructor for Box.
  Box() {
    System.out.println("Constructing Box");
    width = 10;
    height = 10;
    depth = 10;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
class BoxDemo6 {
  public static void main(String args[]) {
    // declare, allocate, and initialize Box objects
    Box mybox1 = new Box();
    Box mybox2 = new Box();

    double vol;

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

When this program is run, it generates the following results:

```
Constructing Box
Constructing Box
Volume is 1000.0
Volume is 1000.0
```

As you can see, both **mybox1** and **mybox2** were initialized by the **Box( )** constructor when they were created. Since the constructor gives all boxes the samedimensions, 10 by 10 by 10, both **mybox1** and **mybox2** will have the same volume.

Most constructors will not display anything. They will simply initialize an object.

Parameterized Constructors

While the **Box( )** constructor in the preceding example does initialize a **Box** object, it is not very useful—all boxes have the same dimensions. What is needed is a way to construct **Box** objects of various dimensions. The easy solution is to add parameters to the constructor. For example, the following version of **Box** defines a parameterized constructor that sets the dimensions of a box as specified by those parameters.

```
/* Here, Box uses a parameterized constructor to
   initialize the dimensions of a box.
*/
class Box {
  double width;
  double height;
  double depth;

  // This is the constructor for Box.
  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
class BoxDemo7 {
  public static void main(String args[]) {
    // declare, allocate, and initialize Box objects
    Box mybox1 = new Box(10, 20, 15);
    Box mybox2 = new Box(3, 6, 9);

    double vol;

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

The output from this program is shown here:

```
Volume is 3000.0
Volume is 162.0
```

As you can see, each object is initialized as specified in the parameters to its constructor. For example, in the following line,

```
Box mybox1 = new Box(10, 20, 15);
```
the values 10, 20, and 15 are passed to the **Box( )** constructor when **new** creates the object. Thus, **mybox1**'s copy of **width, height**, and **depth** will contain the values 10, 20, and 15, respectively.

THE "this" KEYWORD

Java defines the "**this**"keyword. "**this**"can be used inside any method to refer to the *current* object. That is, **this** is always a reference to the object on which the method was invoked.

Consider the following version of **Box( )**:

```
// A redundant use of this.
Box(double w, double h, double d) {
  this.width = w;
  this.height = h;
  this.depth = d;
}
```

This version of **Box( )** operates exactly like the earlier version. Inside **Box( ), this** will always refer to the invoking object.

Instance Variable Hiding

As you know, it is illegal in Java to declare two local variables with the same name inside the same or enclosing scopes. Interestingly, you can have local variables,including formal parameters to methods, which overlap with the names of the class' instance variables. However, when a local variable has the same name as an instance variable, the local variable *hides* the instance variable.

For example, here is another version of **Box( )**, which uses **width, height**, and **depth** for parameter names and then uses **this** to access the instance variables by the same name:

```
// Use this to resolve name-space collisions.
Box(double width, double height, double depth) {
  this.width = width;
  this.height = height;
  this.depth = depth;
}
```

OVERLOADING METHODS

In Java, it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different. When this is the case, the methods are said to be overloaded, and the process is referred to as *methodoverloading*. Method overloading is one of the ways that Java supports polymorphism.

When an overloaded method is invoked, Java uses the type and/or number of arguments as its guide to determine which version of the overloaded method to

actually call. Thus, overloaded methods must differ in the type and/or number of their parameters. While overloaded methods may have different return types, the return type alone is insufficient to distinguish two versions of a method. When Java encounters a call to an overloaded method, it simply executes the version of the method whose parameters match the arguments used in the call.
Here is a simple example that illustrates method overloading:

```
// Demonstrate method overloading.
class OverloadDemo {
  void test() {
    System.out.println("No parameters");
  }

  // Overload test for one integer parameter.
  void test(int a) {
    System.out.println("a: " + a);
  }
  // Overload test for two integer parameters.
  void test(int a, int b) {
    System.out.println("a and b: " + a + " " + b);
  }
  // Overload test for a double parameter
  double test(double a) {
    System.out.println("double a: " + a);
    return a*a;
  }
}

class Overload {
  public static void main(String args[]) {
    OverloadDemo ob = new OverloadDemo();
    double result;

    // call all versions of test()
    ob.test();
    ob.test(10);
    ob.test(10, 20);
    result = ob.test(123.25);
    System.out.println("Result of ob.test(123.25): " + result);
  }
}
```

This program generates the following output:

```
No parameters
a: 10
a and b: 10 20
double a: 123.25
Result of ob.test(123.25): 15190.5625
```

As you can see, **test( )** is overloaded four times. The first version takes no parameters, the second takes one integer parameter, the third takes two integer parameters, and the fourth takes one **double** parameter.

When an overloaded method is called, Java looks for a match between the arguments used to call the method and the method's parameters. However, this match need not always be exact. In some cases, Java's automatic type conversions can play a role in overload resolution. For example, consider the following program:

```
// Automatic type conversions apply to overloading.
class OverloadDemo {
  void test() {
    System.out.println("No parameters");
  }

  // Overload test for two integer parameters.
  void test(int a, int b) {
    System.out.println("a and b: " + a + " " + b);
  }

  // Overload test for a double parameter
  void test(double a) {
    System.out.println("Inside test(double) a: " + a);
  }
}

class Overload {
  public static void main(String args[]) {
    OverloadDemo ob = new OverloadDemo();
    int i = 88;

    ob.test();
    ob.test(10, 20);

    ob.test(i); // this will invoke test(double)
    ob.test(123.2); // this will invoke test(double)
  }
}
```

This program generates the following output:

```
No parameters
a and b: 10 20
Inside test(double) a: 88
Inside test(double) a: 123.2
```

As you can see, this version of **OverloadDemo**does not define **test(int)**. Therefore, when **test( )** is called with an integer argument inside **Overload**, no matching method is found. However, Java can automatically convert an integer into a **double**, and this conversion can be used to resolve the call. Therefore, after **test(int)** is not found, Java elevates **i** to **double** and then calls **test(double)**. Java will employ its automatic type conversions only if no exact match is found.

Overloading Constructors

In addition to overloading normal methods, you can also overload constructor methods.

Following is the latest version of **Box**:

```
class Box {
  double width;
  double height;
  double depth;

  // This is the constructor for Box.
  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

As you can see, the **Box( )** constructor requires three parameters. This means that all declarations of **Box** objects must pass three arguments to the **Box( )** constructor. For example, the following statement is currently invalid:

```
Box ob = new Box();
```

Since **Box( )** requires three arguments, it's an error to call it without them.

The solution to these problems is quite easy: simply overload the **Box** constructor so that it handles the situations just described. Here is a program thatcontains an improved version of **Box** that does just that:

```
/* Here, Box defines three constructors to initialize
   the dimensions of a box various ways.
*/
class Box {
  double width;
  double height;
  double depth;

  // constructor used when all dimensions specified
  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }

  // constructor used when no dimensions specified
  Box() {
    width = -1;  // use -1 to indicate
    height = -1; // an uninitialized
    depth = -1;  // box
  }

  // constructor used when cube is created
  Box(double len) {
    width = height = depth = len;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}

class OverloadCons {
  public static void main(String args[]) {
    // create boxes using the various constructors
    Box mybox1 = new Box(10, 20, 15);
    Box mybox2 = new Box();
    Box mycube = new Box(7);

    double vol;
```

```
    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume of mybox1 is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume of mybox2 is " + vol);

    // get volume of cube
    vol = mycube.volume();
    System.out.println("Volume of mycube is " + vol);
  }
}
```

The output produced by this program is shown here:

```
Volume of mybox1 is 3000.0
Volume of mybox2 is -1.0
Volume of mycube is 343.0
```

As you can see, the proper overloaded constructor is called based upon the arguments specified when **new** is executed.

USING OBJECTS AS PARAMETERS

So far, we have only been using simple types as parameters to methods. However, it is both correct and common to pass objects to methods. For example, consider the following short program:

```
// Objects may be passed to methods.
class Test {
  int a, b;

  Test(int i, int j) {
    a = i;
    b = j;
  }

  // return true if o is equal to the invoking object
  boolean equalTo(Test o) {
    if(o.a == a && o.b == b) return true;
    else return false;
  }
}
```

```
class PassOb {
  public static void main(String args[]) {
    Test ob1 = new Test(100, 22);
    Test ob2 = new Test(100, 22);
    Test ob3 = new Test(-1, -1);

    System.out.println("ob1 == ob2: " + ob1.equalTo(ob2));
    System.out.println("ob1 == ob3: " + ob1.equalTo(ob3));
  }
}
```

This program generates the following output:

```
ob1 == ob2: true
ob1 == ob3: false
```

As you can see, the **equalTo( )** method inside **Test** compares two objects for equality and returns the result. That is, it compares the invoking object with the one that it is passed. If they contain the same values, then the method returns **true**. Otherwise, it returns **false**. Notice that the parameter **o** in **equalTo( )** specifies **Test**as its type.

RETURNING OBJECTS

A method can return any type of data, including class types that you create. For example, in the following program, the **incrByTen( )** method returns an object in which the value of **a** is ten greater than it is in the invoking object.

```
// Returning an object.
class Test {
  int a;

  Test(int i) {
    a = i;
  }

  Test incrByTen() {
    Test temp = new Test(a+10);
    return temp;
  }
}

class RetOb {
  public static void main(String args[]) {
    Test ob1 = new Test(2);
    Test ob2;
```

```
    ob2 = ob1.incrByTen();
    System.out.println("ob1.a: " + ob1.a);
    System.out.println("ob2.a: " + ob2.a);
    ob2 = ob2.incrByTen();
    System.out.println("ob2.a after second increase: "
                       + ob2.a);
  }
}
```

The output generated by this program is shown here:

```
ob1.a: 2
ob2.a: 12
ob2.a after second increase: 22
```

As you can see, each time **incrByTen( )** is invoked, a new object is created, and a reference to it is returned to the calling routine.

RECURSION

Java supports *recursion*. Recursion is the process of defining something in terms of itself. As it relates to Java programming, recursion is the attribute that allows a method to call itself. A method that calls itself is said to be *recursive*.

The classic example of recursion is the computation of the factorial of a number. The factorial of a number *N* is the product of all the whole numbers between 1 and *N*. For example, 3 factorial is 1 × 2 × 3 ×, or 6. Here is how a factorial can be computed by use of a recursive method:

```
// A simple example of recursion.
class Factorial {
  // this is a recursive method
  int fact(int n) {
    int result;

    if(n==1) return 1;
    result = fact(n-1) * n;
    return result;
  }
}

class Recursion {
  public static void main(String args[]) {
    Factorial f = new Factorial();
```

```
    System.out.println("Factorial of 3 is " + f.fact(3));
    System.out.println("Factorial of 4 is " + f.fact(4));
    System.out.println("Factorial of 5 is " + f.fact(5));
  }
}
```

The output from this program is shown here:

```
Factorial of 3 is 6
Factorial of 4 is 24
Factorial of 5 is 120
```

UNDERSTANDING "static"

There will be times when you will want to define a class member that will be used independently of any object of that class. Normally, a class member must be accessed only with an object of its class. However, it is possible to create a member that can be used by itself, without reference to a specific instance.

To create such a member, precede its declaration with the keyword **static**. When a member is declared **static**, it can be accessed before any objects of its class are created, and without reference to any object. You can declare both methods and variables to be **static**. The most common example of a **static** member is **main( )**. **main( )** is declared as **static** because it must be called before any objects exist.

Instance variables declared as **static** are, essentially, global variables.

Methods declared as **static** have several restrictions:

- They can only directly call other **static** methods of their class.
- They can only directly access **static** variables of their class.
- They cannot refer to **this** or **super** in any way.

If you need to do computation in order to initialize your **static** variables, you candeclare a **static** block that gets executed exactly once, when the class is first loaded.

The following example shows a class that has a **static** method, some **static** variables, and a **static** initialization block:

```
// Demonstrate static variables, methods, and blocks.
class UseStatic {
  static int a = 3;
  static int b;
```

```
  static void meth(int x) {
    System.out.println("x = " + x);
    System.out.println("a = " + a);
    System.out.println("b = " + b);
  }

  static {
    System.out.println("Static block initialized.");
    b = a * 4;
  }

  public static void main(String args[]) {
    meth(42);
  }
}
```

As soon as the **UseStatic**class is loaded, all of the **static** statements are run. First, **a** is set to **3**, then the **static** block executes, which prints a message and then initializes **b** to **a*4** or **12**. Then **main( )** is called, which calls **meth( )**, passing **42** to **x**. The three **println( )** statements refer to the two **static** variables **a** and **b**, as well as to the parameter **x**.

Here is the output of the program:

```
Static block initialized.
x = 42
a = 3
b = 12
```

Outside of the class in which they are defined, **static** methods and variables can be used independently of any object. To do so, you need only specify the name of their class followed by the dot operator. For example, if you wish to call a **static**method from outside its class, you can do so using the following general form:

`classname.method( )`

Here, *classname*is the name of the class in which the **static** method is declared. Here is an example. Inside **main( )**, the **static** method **callme( )** and the **static** variable **b** are accessed through their class name **StaticDemo**.

```
class StaticDemo {
  static int a = 42;
  static int b = 99;

  static void callme() {
    System.out.println("a = " + a);
  }
}

class StaticByName {
  public static void main(String args[]) {
    StaticDemo.callme();
    System.out.println("b = " + StaticDemo.b);
  }
}
```

Here is the output of this program:

```
a = 42
b = 99
```

INTRODUCING "final"

A field can be declared as **final**. Doing so prevents its contents from being modified, making it, essentially, a constant. This means that you must initialize a **final** field when it is declared. You can do this in one of two ways: First, you can give it a value when it is declared. Second, you can assign it a value within a constructor. The first approach is the most common. Here is an example:

```
final int FILE_NEW = 1;
final int FILE_OPEN = 2;
final int FILE_SAVE = 3;
final int FILE_SAVEAS = 4;
final int FILE_QUIT = 5;
```

It is a common coding convention to choose all uppercase identifiers for **final** fields, as this example shows.

In addition to fields, both method parameters and local variables can be declared **final**. Declaring a parameter **final** prevents it from being changed within the method.

Declaring a local variable **final** prevents it from being assigned a value more than once.

The keyword **final** can also be applied to methods

INTRODUCING NESTED AND INNER CLASSES

It is possible to define a class within another class; such classes are known as *nested classes*. The scope of a nested class is bounded by the scope of its enclosing class.

A nested class has access to the members, including private members, of the class in which it is nested. However, the enclosing class does not have access to the members of the nested class.

There are two types of nested classes: *static* and *non-static*. A static nested class is one that has the **static** modifier applied. Because it is static, it must access the non-static members of its enclosing class through an object. That is, it cannot refer to non-static members of its enclosing class directly.

The most important type of nested class is the *inner* class. An inner class is non-staticand may refer to them directly in the same way that other non-static members of the outer class do.

The following program illustrates how to define and use an inner class. The class named **Outer** has one instance variable named **outer_x**, one instance method named **test( )**, and defines one inner class called **Inner**.

```
// Demonstrate an inner class.
class Outer {
  int outer_x = 100;

  void test() {
    Inner inner = new Inner();
    inner.display();
  }

  // this is an inner class
  class Inner {
    void display() {
      System.out.println("display: outer_x = " + outer_x);
    }
  }
}
```

```
class InnerClassDemo {
  public static void main(String args[]) {
    Outer outer = new Outer();
    outer.test();
  }
}
```

Output from this application is shown here:

```
display: outer_x = 100
```

In the program, an inner class named **Inner** is defined within the scope of class **Outer**. Therefore, any code in class **Inner** can directly access the variable **outer_x**. An instance method named **display( )** is defined inside **Inner**. This method displays**outer_x**on the standard output stream. The **main( )** method of **InnerClassDemo**creates an instance of class **Outer** and invokes its **test( )** method. That methodcreates an instance of class **Inner** and the **display( )** method is called. An inner class has access to all of the members of its enclosingclass, but the reverse is not true. Members of the inner class are known only withinthe scope of the inner class and may not be used by the outer class. For example,

```
// This program will not compile.
class Outer {
  int outer_x = 100;

  void test() {
    Inner inner = new Inner();
    inner.display();
  }

  // this is an inner class
  class Inner {
    int y = 10; // y is local to Inner

    void display() {
      System.out.println("display: outer_x = " + outer_x);
    }
  }

  void showy() {
    System.out.println(y); // error, y not known here!
  }
}
```

```
class InnerClassDemo {
  public static void main(String args[]) {
    Outer outer = new Outer();
    outer.test();
  }
}
```

Here, **y** is declared as an instance variable of **Inner**. Thus, it is not known outsideof that class and it cannot be used by **showy( )**.

It is possible to define inner classes within any block scope. Forexample, you can define a nested class within the block defined by a method or evenwithin the body of a **for**loop, as this next program shows:

```
// Define an inner class within a for loop.
class Outer {
  int outer_x = 100;

  void test() {
    for(int i=0; i<10; i++) {
      class Inner {
        void display() {
          System.out.println("display: outer_x = " + outer_x);
        }
      }
      Inner inner = new Inner();
      inner.display();
    }
  }
}

class InnerClassDemo {
  public static void main(String args[]) {
    Outer outer = new Outer();
    outer.test();
  }
}
```

The output from this version of the program is shown here:

```
display: outer_x = 100
display: outer_x = 100
display: outer_x = 100
display: outer_x = 100
display: outer_x = 100
display: outer_x = 100
display: outer_x = 100
display: outer_x = 100
display: outer_x = 100
display: outer_x = 100
```

USING COMMAND-LINE ARGUMENTS

Sometimes you will want to pass information into a program when you run it. This is accomplished by passing *command-line arguments* to **main( )**. A command-lineargument is the information that directly follows the program's name on the command line when it is executed. To access the command-line arguments inside a Java program is quite easy—they are stored as strings in a **String** array passed to the **args**parameter of **main( )**. The first command-line argument is stored at **args[0]**, the second at **args[1]**, and so on. For example, the following program displays all of the command-line arguments that it is called with:

```
// Display all command-line arguments.
class CommandLine {
  public static void main(String args[]) {
    for(int i=0; i<args.length; i++)
      System.out.println("args[" + i + "]: " +
                         args[i]);
  }
}
```

Try executing this program, as shown here:

```
java CommandLine this is a test 100 -1
```

When you do, you will see the following output:

```
args[0]: this
args[1]: is
args[2]: a
args[3]: test
args[4]: 100
args[5]: -1
```

VARARGS: VARIABLE-LENGTH ARGUMENTS

Beginning with JDK 5, Java has included a feature that simplifies the creation of methods that need to take a variable number of arguments. This feature is called *varargs*and it is short for *variable-length arguments*. A method that takes a variable number of arguments is called a *variable-arity method*, or simply a *varargs method*.

A variable number of arguments be passed to a method are not unusual.

Prior to JDK 5, variable-length arguments could be handled two ways. First, if the maximum number of arguments was small and known, then you could create overloaded versions of the method, one for each way the method could be called.In cases where the maximum number of potential arguments was larger, or unknowable, a second approach was used in which the arguments were put into an array, and then the array was passed to the method. This approach is illustrated by the following program:

```
// Use an array to pass a variable number of
// arguments to a method. This is the old-style
// approach to variable-length arguments.
class PassArray {
  static void vaTest(int v[]) {
    System.out.print("Number of args: " + v.length +
                     " Contents: ");

    for(int x : v)
      System.out.print(x + " ");
    System.out.println();
  }

  public static void main(String args[])
  {
    // Notice how an array must be created to
    // hold the arguments.
    int n1[] = { 10 };
    int n2[] = { 1, 2, 3 };
    int n3[] = { };

    vaTest(n1); // 1 arg
    vaTest(n2); // 3 args
    vaTest(n3); // no args
  }
}
```

The output from the program is shown here:

```
Number of args: 1 Contents: 10
Number of args: 3 Contents: 1 2 3
Number of args: 0 Contents:
```

In the program, the method **vaTest( )** is passed its arguments through the array **v**. This old-style approach to variable-length arguments does enable **vaTest( )** to take an arbitrary number of arguments. However, it requires that these arguments be manually packaged into an array prior to calling **vaTest( )**. It is tedious to construct an array each time **vaTest( )** is called. The varargs feature offers a simpler, better option.

A variable-length argument is specified by three periods (**…**). For example, hereis how **vaTest( )** is written using a vararg:

```
static void vaTest(int ... v) {
```

This syntax tells the compiler that **vaTest( )** can be called with zero or more arguments. As a result, **v** is implicitly declared as an array of type **int[ ]**. Thus, inside **vaTest( ), v** is accessed using the normal array syntax. Here is the precedingprogram rewritten using a vararg:

```
// Demonstrate variable-length arguments.
class VarArgs {

  // vaTest() now uses a vararg.
  static void vaTest(int ... v) {
    System.out.print("Number of args: " + v.length +
                     " Contents: ");

    for(int x : v)
      System.out.print(x + " ");

    System.out.println();
  }

  public static void main(String args[])
  {
    // Notice how vaTest() can be called with a
    // variable number of arguments.
    vaTest(10);      // 1 arg
    vaTest(1, 2, 3); // 3 args
    vaTest();        // no args
  }
}
```

The output from the program is the same as the original version.

There are two important things to notice about this program. First, as explained, inside **vaTest( ), v** is operated on as an array. This is because **v** *is* an array. The … syntax simply tells the compiler that a variable number of arguments will be used, and that these arguments will be stored in the array referred to by **v**. Second, in **main( ), vaTest( )** is called with different numbers of arguments, including no arguments at all. The arguments are automatically put in an array and passed to **v**. In the case of no arguments, the length of the array is zero.

A method can have "normal" parameters along with a variable-length parameter. However, the variable-length parameter must be the last parameter declared by the method. For example, this method declaration is perfectly acceptable:

```
int doIt(int a, int b, double c, int ... vals) {
```

In this case, the first three arguments used in a call to **doIt( )** are matched to the first three parameters. Then, any remaining arguments are assumed to belong to **vals**. Remember, the varargs parameter must be last. For example, the following declaration is incorrect:

```
int doIt(int a, int b, double c, int ... vals, boolean stopFlag) { // Error!
```

Here, there is an attempt to declare a regular parameter after the varargs parameter, which is illegal.

There is one more restriction to be aware of: there must be only one varargsparameter. For example, this declaration is also invalid:

```
int doIt(int a, int b, double c, int ... vals, double ... morevals) { // Error!
```

The attempt to declare the second varargs parameter is illegal.

Here is a reworked version of the **vaTest( )** method that takes a regular argument and a variable-length argument:

```
// Use varargs with standard arguments.
class VarArgs2 {

  // Here, msg is a normal parameter and v is a
  // varargs parameter.
  static void vaTest(String msg, int ... v) {
    System.out.print(msg + v.length +
                     " Contents: ");

    for(int x : v)
      System.out.print(x + " ");

    System.out.println();
  }
```

```
  public static void main(String args[])
  {
    vaTest("One vararg: ", 10);
    vaTest("Three varargs: ", 1, 2, 3);
    vaTest("No varargs: ");
  }
}
```

The output from this program is shown here:

```
One vararg: 1 Contents: 10
Three varargs: 3 Contents: 1 2 3
No varargs: 0 Contents:
```

Overloading Vararg Methods

You can overload a method that takes a variable-length argument. For example, the following program overloads **vaTest( )** three times:

```
// Varargs and overloading.
class VarArgs3 {

  static void vaTest(int ... v) {
    System.out.print("vaTest(int ...): " +
                     "Number of args: " + v.length +
                     " Contents: ");

    for(int x : v)
      System.out.print(x + " ");

    System.out.println();
  }

  static void vaTest(boolean ... v) {
    System.out.print("vaTest(boolean ...) " +
                     "Number of args: " + v.length +
                     " Contents: ");

    for(boolean x : v)
      System.out.print(x + " ");

    System.out.println();
  }
```

```
  static void vaTest(String msg, int ... v) {
    System.out.print("vaTest(String, int ...): " +
                     msg + v.length +
                     " Contents: ");

    for(int x : v)
      System.out.print(x + " ");

    System.out.println();
  }

  public static void main(String args[])
  {
    vaTest(1, 2, 3);
    vaTest("Testing: ", 10, 20);
    vaTest(true, false, false);
  }
}
```

The output produced by this program is shown here:

```
vaTest(int ...): Number of args: 3 Contents: 1 2 3
vaTest(String, int ...): Testing: 2 Contents: 10 20
vaTest(boolean ...) Number of args: 3 Contents: true false false
```

This program illustrates both ways that a varargs method can be overloaded. First, the types of its vararg parameter can differ. This is the case for **vaTest(int …)** and **vaTest(boolean …)**. Remember, the **…** causes the parameter to be treated as an array of the specified type. Therefore, just as you can overload methods by using different types of array parameters, you can overload vararg methods by using different types of varargs. In this case, Java uses the type difference to determine which overloaded method to call.

The second way to overload a varargs method is to add one or more normal parameters. This is what was done with **vaTest(String, int …)**. In this case, Java uses both the number of arguments and the type of the arguments to determine which method to call.

Varargs and Ambiguity

Somewhat unexpected errors can result when overloading a method that takes a variable-length argument. These errors involve ambiguity because it is possible to create an ambiguous call to an overloaded varargs method. For example, consider the following program:

```
// Varargs, overloading, and ambiguity.
//
// This program contains an error and will
// not compile!
class VarArgs4 {

  static void vaTest(int ... v) {
    System.out.print("vaTest(int ...): " +
                     "Number of args: " + v.length +
                     " Contents: ");

    for(int x : v)
      System.out.print(x + " ");

    System.out.println();
  }

  static void vaTest(boolean ... v) {
    System.out.print("vaTest(boolean ...) " +
                     "Number of args: " + v.length +
                     " Contents: ");

    for(boolean x : v)
      System.out.print(x + " ");

    System.out.println();
  }

  public static void main(String args[])
  {
    vaTest(1, 2, 3); // OK
    vaTest(true, false, false); // OK
    vaTest(); // Error: Ambiguous!
  }
}
```

In this program, the overloading of **vaTest( )** is perfectly correct. However, this program will not compile because of the following call:

```
vaTest(); // Error: Ambiguous!
```

Because the vararg parameter can be empty, this call could be translated into a call to **vaTest(int …)** or **vaTest(boolean …)**. Both are equally valid. Thus, the call is inherently ambiguous.

Chapter 3
Arrays and Strings

INTRODUCTION

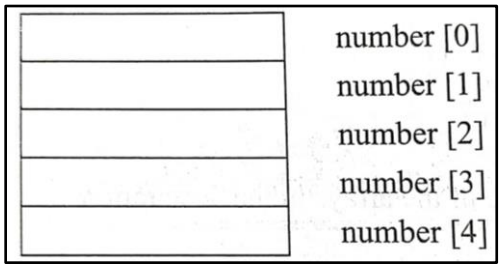
An array is a group of contiguous or related data items that share a common name. For instance, we can define an array name **salary** to represent a set of salaries of a group of employees. A particular value is indicated by writing a number called **index** number or **subscript** in brackets after the array name. For example `salary [10]` represents the salary of the 10^{th} employee. While the complete set of values is referred to as an array, the individual values are called elements. Arrays can be of any variable type.

ONE-DIMENSIONAL ARRAYS

A list of items can be given one variable name using only one subscript and such a variable is called a **single-subscripted** variable or a **one-dimensional** array. The subscript begins with a number 0. That is `x[0]` is allowed. For example, if we want to represent a set of 5 numbers, say (35, 40, 20, 57, 19), by an array variable number, then we may create the variable number as follows:

```
int number[]=new int[5];
```

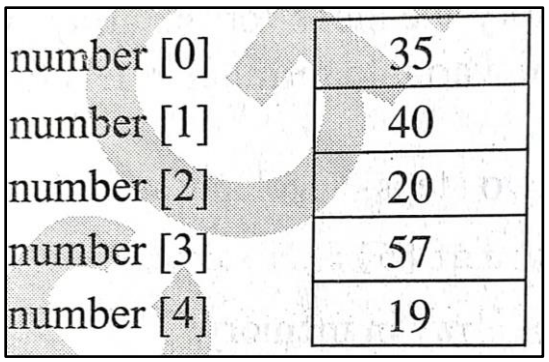
And the computer reserves 5 storage locations as shown below:



The values to the array elements can be assigned as follows:

```
number[0]=35;
number[1]=40;
number[2]=20;
number[3]=57;
number[4]=19;
```

This would cause the array number to store the values shown as follows:



CREATING AN ARRAY

Like other variables, arrays must be declared and created in the computer memory before they are used. Creation of an array involves three steps:

1. Declaring the array
2. Creating memory locations
3. Putting values into the memory locations

Declaration of Arrays

Arrays in Java may be declared in two forms:

Form1

```
type arrayname[];
```

Form2

```
type [] arrayname;
```

Examples:

```
int number[];
float average[];
int [] counter;
float[] marks;
```

We do not enter the size of the arrays in the declaration.

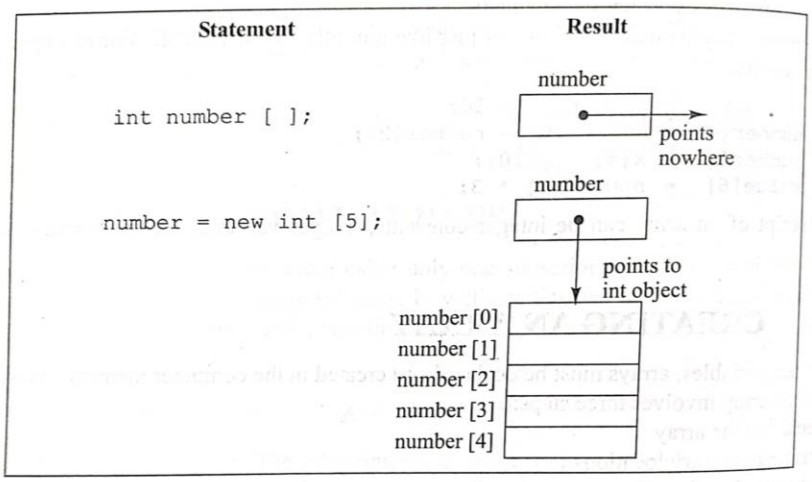
Creation of Arrays

After declaring an array, we need to create it in the memory. Java allows us to create arrays using **new** operator as shown below:

```
arrayname=new type[size];
```

Examples:

```
number=new int[5];
average=new float[10];
```
These lines create necessary memory locations for the arrays **number** and **average** and designate them as **int** and **float** respectively. Now, the variable **number** refers to an array of 5 integers and **average** refers to an array of 10 floating point values.
It is possible to combine the two steps- declaration and creation- into one as shown as below:
```
int number[]=new int[5];
```
The below figure illustrates creation of an array in memory.



Initialization of Arrays

The final step is to put values into the array created. This process is known as **initialization**. This is done using the array subscripts as shown below:

```
arrayname [subscript]=value;
```

Example:
```
number[0]=35;
number[1]=40;
number[2]=20;
number[3]=57;
number[4]=19;
```
Java creates arrays starting with the subscript of 0 and ends with a value one less than the size specified.

Java protects arrays from overruns and underruns. Trying to access an array bound its boundaries will generate an error message.
We can initialize arrays automatically in the same way as the ordinary variables when they are declared, as shown below:

```
type arrayname[]={list of values};
```

The array initializer is a list of values separated by commas and surrounded by curly braces. The compiler allocates enough space for all the elements specified in the list.
Example:

```
int number []={35,40,20,57,19};
```

It is possible to assign an array object to another. For example:

```
int a[]={1,2,3};
int b[];
```

`b=a;`are valid in Java. Both the arrays will have the same values.
Loops may be used to initialize large size arrays. For example:

```
.....
.....
for(int i=0;i<100;i++)
{
     if (i<50)
          sum[i]=0.0;
     else
          sum[i]=1.0;
}
.....
.....
```

The first fifty elements of the array **sum** are initialized to zero while the remaining are initialized to 1.0.

Array Length

In Java, all arrays store the allocated size in a variable named **length**. We can obtain the length of the array **a** using **a.length**. Example:

```
int aSize=a.length;
```

This information will be useful in the manipulation of arrays when their sizes are not known. The below program illustrates the use of an array for sorting a list of numbers.

```
class NumberSorting
{
public static void main(String args[])
{
int number[]={55,40,80,65,71};
int n=number.length;
System.out.println("Given list:");
for(int i=0;i<n;i++)
System.out.println(number[i]);
//sorting
for(int i=0;i<n;i++)
{
for(int j=i+1;j<n;j++)
{
if(number[i]<number[j])
{
//interchange values
int temp=number[i];
number[i]=number[j];
number[j]=temp;
}
}
}//sorting ends
System.out.println("Sorted list:");
for(int i=0;i<n;i++)
System.out.println(number[i]);
}
}
```

OUTPUT

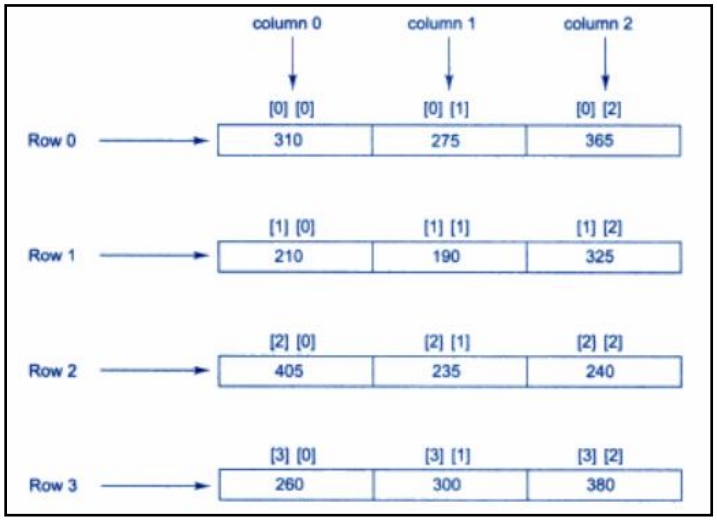
```
Given list:
55
40
80
65
71
Sorted list:
80
71
65
55
40
```

TWO-DIMENSIONAL ARRAYS

Consider the following data table, which shows the value of sales of 3 items by 4 sales girls:

	Item1	Item2	Item3
Salesgirl #1	310	275	365
Salesgirl #2	210	190	325
Salesgirl #3	405	235	240
Salesgirl #4	260	300	380

The table contains a total of 12 values, three in each line. We can think of this table as a matrix consisting of 4 rows and 3 columns. Each row represents the values of sales by a particular salesgirl and each column represents the values of sales of a particular item.
In mathematics, we represent a particular value in a matrix by using two subscripts such as v_{ij}. Here v denotes the entire matrix and v_{ij} refers to the value in the i^{th} row and j^{th} column. For example, in the above table v_{23} refers to the value 325. Java allows us to define such tables of items by using **two-dimensional** arrays. The table discussed above can be represented in Java as:
`v[4][3]`
Two dimensional arrays are stored in memory as shown in below figure. As with the single dimensional arrays, each dimension of the array is indexed from zero to its maximum size minus one. With two-dimensional array, the first index selects the row and the second index selects the column within that row.



For creating a two-dimensional array, we must follow the same steps as that of simple arrays. We may create a two-dimensional array like this:

```
int myArray[][];
myArray=new int[3][4];
```

Or

```
int myArray[][]=new int[3][4];
```

This creates a table that can store 12 integer values, four across and three down. Like the one-dimensional arrays, two-dimensional arrays may be initialized by following the declaration with a list of initial values enclosed in braces. For example, `int table[2][3]={0,0,0,1,1,1};`
Initializes the elements of the first row to 0 and the second row to 1. The initialization is done row by row. The above statement can be equivalently written as: `int table[][]={0,0,0},{1,1,1};`

We can also initialize a two-dimensional array on the form of a matrix as shown below:

```
int table[][]={
                    {0,0,0},
                    {1,1,1}
          };
```

We can refer to a value stored in a two-dimensional array by using subscripts for both the column and row of the corresponding element. For example,

```
int value=table[1][2];
```

This retrieves the value stored in the second row and third column of a table matrix.

A quick way to initialize a two dimensional array is to use nested for loops as shown below:

```
for(i=0;i<5;i++)
{
for(j=0;j<5;j++)
{
if(i==j)
table[i][j]=1;
else
table[i][j]=0;
}
}
```

This will set all the diagonal elements to 1 and others to zero as given below:

1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

```
// Demonstrate a two-dimensional array.
class TwoDArray {
  public static void main(String args[]) {
    int twoD[][]= new int[4][5];
    int i, j, k = 0;

    for(i=0; i<4; i++)
      for(j=0; j<5; j++) {
        twoD[i][j] = k;
        k++;
      }

    for(i=0; i<4; i++) {
      for(j=0; j<5; j++)
        System.out.print(twoD[i][j] + " ");
      System.out.println();
    }
  }
}
```

Output:

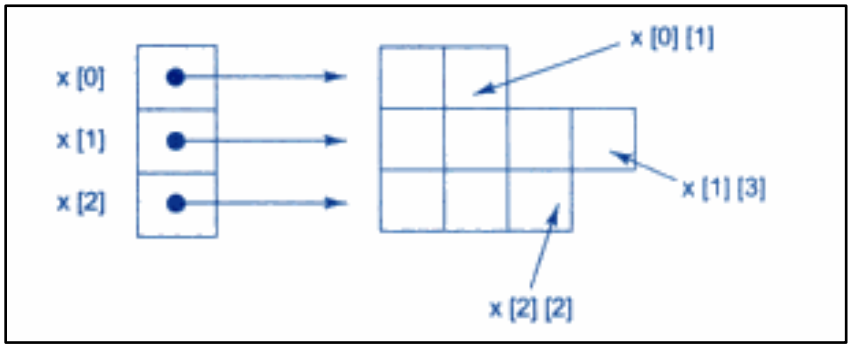
```
 0  1  2  3  4
 5  6  7  8  9
 10 11 12 13 14
 15 16 17 18 19
```

Variable Size Arrays

Java treats multidimensional array as "array of arrays". It is possible to declare a two dimensional array as follows:

```
int x[][]=new int[3][];
x[0]=new int[2];
x[1]=new int[4];
x[2]=new int[3];
```

These statements create a two-dimensional array as having different lengths for each row as shown in below figure:



STRINGS

Strings represent a sequence of characters. The easiest way to represent a sequence of characters in Java is by using a character array. Example:

```
char charArray[]=new char[4];
charArray[0]='J';
charArray[1]='a';
charArray[2]='v';
charArray[3]='a';
```

In Java, strings are class objects and implemented using two classes, namely, **String** and **StringBuffer**. A Java string is an instantiated object of the **String** class. Java strings are more reliable and predictable. Strings may be declared and created as follows:

```
String stringName;
stringName=new String("string");
```

Example:

```
String firstName;
firstName=new String ("Anil");
```

These two statements may be combined as follows:

```
String firstName=new String ("Anil");
```

Like arrays, it is possible to get the length of string using **length** method of the **String** class. Example:

```
int m=firstName.length();
```

Java strings can be concatenated using the + operator. For example:

```
String fullName=name1+name2;
String city1= "New"+ "Delhi";
```

Where **name1** and **name2** are Java strings containing string constants. Another example is:

```
System.out.println(firstName+ "Kumar");
```

String Arrays

We can also create and use arrays that contain strings. The statement

```
String itemArray[]=new String[3];
```

Will create an itemArray of size 3 to hold three string constants. We can assign the strings to the itemArray element by element using three different statements or more efficiently using a **for** loop.

String Methods

The String class defines a number of methods that allow us to accomplish a variety of string manipulation tasks. The below table lists some of the most commonly used string methods, and their tasks.

Some most commonly used string methods are given below

Method call	Task performed
s2=s1.toLowerCase;	Converts the strings to all lowercase
s2=s1.toUpperCase;	Converts the strings to all uppercase
s2=s1.replace('x','y');	Replace all appearances of x with y
s2=s1.trim();	Remove white spaces at the beginning and end of the string s1
s1.equals(s2)	Returns 'true' if s1 is equal to s2
s1.equalsIgnoreCase(s2)	Returns 'true' if s1=s2, ignoring the case of characters
s1.length()	Gives the length of s1
s1.charAt(n)	Gives n^{th} character of s1
s1.compareTo(s2)	Returns negative if s1>s2, positive if s1>s2, and zero if s1 is equal to s2
s1.concat(s2)	Concatenates s1 and s2
s1.substring(n)	Gives substring starting from n^{th} character
s1.substring(n,m)	Gives substring starting from n^{th} character up to m^{th} (not including m^{th})
String.Valueof(p)	Creates a string object of the parameter
s1.toString()	Creates a string representation of the object p
s1.indexof('x')	Gives the position of the first occurrence of 'x' in the string s1
s1.indexof('x',n)	Gives the position of 'x' that occurs after nth position in the string s1
String.Valueof(Variabl;e)	Converts the parameter value to string representation

```
public class Stringoperation1
{
public static void main(String args[])
{
String s="Sachin";
System.out.println(s.toUpperCase());
System.out.println(s.toLowerCase());
System.out.println(s.charAt(0));
System.out.println(s.charAt(3));
```

```
System.out.println(s.length());
int a=10;
String s1=String.valueOf(a);
System.out.println(s1+10);
String s2=" Sachin ";
System.out.println(s.trim());
}
}
```

Output
SACHIN
sachin
S
h
6
1010
Sachin

StringBuffer Class

StringBuffer is a peer class of **String**. While String creates strings of fixed_length. **StringBuffer** creates strings of flexible length that can be modified in terms of both length and content. We can insert characters and substrings in the middle of a string, or append another string to the end. The below table lists some of the methods that are frequently used in string manipulations.

Method	Task
s1.setCharAt(n,'x')	Modifies the nth character to x
s1.append(s2)	Appends the string s2 to s1 at the end
s1.insert(n,s2)	Inserts the string s2 at the position n of the string s1
s1.setLength(n)	Sets the length of the string s1 to n

VECTORS

Vector class can be used to create a generic dynamic array known as vector that can hold objects of any type and any number. The objects do not have to be homogeneous. Arrays can be easily implemented as vectors. Vectors are created like arrays as follows:

```
Vector intVect=new Vector();//declaring without size
Vector list=new Vector(3);//declaring with size
```

A vector without size can accommodate an unknown number of items. Even, when a size is specified, this can be overlooked and a different number of items may be put into the vector. An array must always have its size specified.
Vectors have a number of advantages over arrays:

1. It is convenient to use vectors to store objects.
2. A vector can be used to store a list of objects that may vary in size.
3. We can add and delete objects from the list as and when required.

A major constraint in using vectors is that we cannot directly store simple data type in a vector, we can only store objects. Therefore, we need to convert simple

types to objects. This can be done using the wrapper classes. The vector class supports a number of methods that can be used to manipulate the vectors created. Important ones are listed below

Method call	Task performed
list.addElement(item)	Adds the item specified to the list at the end
list.elementAt(10)	Gives the name of the 10^{th} object
list.size()	Gives the number of objects present
list.removeElement(item)	Removes the specified item from the list
list.removeElementAt(n)	Removes the item stored in the n^{th} position of the list
list.removeAllElements()	Removes all the elements in the list
list.copy Into(array)	Copies all items from list to array
list.insertElementAt(item,n)	Inserts the item at n^{th} position.

```
import java.util.*;
class LanguageVector
{
public static void main(String args[])
{
Vector list=new Vector();
int length=args.length;
for(int i=0;i<length;i++)
{
list.addElement(args[i]);
}
list.insertElementAt("COBOL",2);
int size=list.size();
String listArray[]=new String[size];
list.copyInto(listArray);
System.out.println("List of languages");
for(int i=0;i<size;i++)
{
System.out.println(listArray[i]);
}
}
}
```

Output

javaLanguageVector Ada BASIC C++ FORTRAN Java
List of languages
Ada
BASIC
COBOL
C++
FORTRAN
Java

WRAPPER CLASSES

Vectors cannot handle primitive data types like int, float, long, char and double. Primitive data types may be converted into object types by using the wrapper classes contained in the **java.lang** package. The below table shows simple data types and their corresponding wrapper class types.

Simple Type	Wrapper Class
boolean	Boolean
char	Character
double	Double
float	Float
int	Integer
long	Long

The wrapper classes have a number of unique methods for handling primitive data types and objects. They are listed in the following tables.

Constructor Calling	Conversion Action
Integer IntVal=new Integer(i);	Primitive integer to Integer objects.
FloatFloatVal=new Float(f);	Primitive float to Float object.
Double DoubleVal=new Double(d);	Primitive double to Double object.
Integer LongVal=new Long(l);	Primitive long to Long object.

The below table shows converting object numbers to primitive numbers using typeValue() method.

Method Calling	Conversion Action
int i=IntVal.intValue();	Object to primitive integer
float f=FloatVal.floatValue();	Object to primitive float
long l=LongVal.longValue();	Object to primitive long
double d=DoubleVal.doubleValue();	Object to primitive doubl

The below table shows converting Numbers to String using String() methods

Method Calling	Conversion Action
str=Integer.toString(i);	Primitive integer to string
str= Float.toString(f);	Primitive float to string
str= Double.toString(d);	Primitive double to string
str= Long.toString(l);	Primitive long to string

The below table shows converting String objects to numeric objects using the Static Method ValueOf()

Method Calling	Conversion Action
DoubleVal=Double.Valueof(str);	Converts string to Double object
FloatVal=Float.Valueof(str);	Converts string to Float object
IntVal=Integer.Valueof(str);	Converts string to Integer object
LongVal=Long.Valueof(str);	Converts string to Long object

The below table shows converting numeric strings to primitive numbers using parsing methods

Method Calling	Conversion Action
int i=Integer.parseInt(str);	Converts string to primitive integer
long i=Long.parseLong(str);	Converts string to primitive long

Autoboxing and Unboxing

The autoboxing and unboxing feature facilitates the process of handling primitive data types in collections. We can use this feature to convert primitive data types to wrapper class types automatically. The compiler generates a code implicitly to convert primitive type to the corresponding wrapper class type and vice versa.
For example, consider the following statements:

```
Double d_object=98.42;
double d_primitive=d_object.doubleValue();
```

Using the autoboxing and unboxing feature, we can rewrite the above code as:

```
Double d_object=98.42;
double  d_primitive=d_object;
```